

Wireless Sensor Network Energy-Aware Routing Protocol

First Review Report

Student: Santhosh

Date: August 24, 2026

Course: Electrical/Electronics Engineering - DSA/Algorithms Project

Project Title: Wireless Sensor Network Energy-Aware Routing Protocol

Abstract

This project implements a complete simulation of an energy-aware routing protocol for Wireless Sensor Networks (WSN) using Data Structures and Algorithms (DSA). The simulation incorporates clustering (LEACH-style), routing (Dijkstra and A* algorithms), and dynamic programming for network lifetime maximization. Implementations are based on the first-order radio model for energy consumption, with all algorithms developed from scratch to demonstrate mastery of DSA concepts. The project includes comprehensive unit testing, configurable simulation parameters, and visualization of results.

1. Objectives

- Implement a WSN simulation using Python that models energy-aware routing protocols.
- Apply and demonstrate core DSA concepts: graph algorithms, clustering, dynamic programming, and heap-based priority queues.
- Develop an energy consumption model based on the first-order radio model (LEACH literature).
- Compare routing algorithms (Dijkstra vs A*) in terms of computational efficiency.
- Implement dynamic programming approach for maximizing network lifetime.
- Generate visualizations and logs for performance analysis.
- Ensure code modularity, readability, and thorough testing.

2. Methodology

2.1 Project Structure

The project follows a modular design with separate files for each major component:

```
wsn-energy-routing/ ■■■■ src/ ■ ■■■■ network.py # Node and graph implementation ■  
■■■■ energy_model.py # Energy consumption model ■ ■■■■ clustering.py # LEACH-style  
clustering ■ ■■■■ routing.py # Dijkstra and A* algorithms ■ ■■■■ dp_lifetime.py #  
Dynamic programming for lifetime maximization ■ ■■■■ simulator.py # Main  
simulation loop ■ ■■■■ visualize.py # Plots and visualizations ■■■■ tests/ # Unit  
test scripts ■■■■ results/ # Output logs and plots ■■■■ requirements.txt #  
Dependencies ■■■■ README.md # Instructions ■■■■ main.py # Entry point
```

2.2 Key Implementations

Network Model (`network.py`)

- **Node Class:** Represents sensor nodes with attributes (ID, position, residual energy, role, cluster ID, alive status).
- **Graph Class:** Uses adjacency list to represent WSN topology. Edge weights are dynamically updated based on the energy model (transmission energy cost).
- **Methods:** Distance calculation, energy consumption, neighbor retrieval, alive node tracking.

Energy Model (`energy\_model.py`)

- Implements the first-order radio model with parameters from LEACH literature:
- Electronics energy (E_{elec}): 50 nJ/bit
- Amplifier energy (E_{amp}): 100 pJ/bit/m²
- Free space and multipath models with threshold distance d_0
- Functions for transmission energy, reception energy, and combined energy cost computation.

Clustering (`clustering.py`)

- **LEACH-style clustering** with probability weighted by residual energy.
- Uses min-heap for efficient selection of cluster-head candidates (tie-breaking by residual energy).
- Assigns member nodes to nearest cluster-head.
- Outputs: Cluster assignment map and list of cluster-heads per round.

Routing (`routing.py`)

- **Dijkstra's Algorithm:** Implemented from scratch using min-heap (heapq). Finds minimum-energy path from cluster-heads to base station (virtual node -1).
- **A* Algorithm:** Uses admissible heuristic (Euclidean distance to base station scaled by electronics energy). Tracks nodes expanded for efficiency comparison.
- **Comparison Module:** Runs both algorithms on the same set of cluster-heads, reporting path equality, execution time, and nodes expanded.
- **Routes Computation:** Function to compute routes for all cluster-heads using selected algorithm.

Dynamic Programming for Lifetime Maximization (`dp\_lifetime.py`)

- **Maximin Path Problem:** Maximizes the minimum residual energy along the path (bottleneck value), subject to hop constraint.
- **DP State:** `dp[node][hops]` = maximum bottleneck energy achievable to reach node using exactly hops`.
- **Recurrence:**
 - `dp[source][0] = residual\_energy[source]`
 - `dp[v][h] = max over neighbors u of { min(dp[u][h-1], residual\_energy[v]) }`
- **Base Station:** Treated as having infinite residual energy.
- **Path Reconstruction:** Uses predecessor table to backtrack optimal path.

Simulator (`simulator.py`)

- **Main Loop** (per round):
 1. Re-cluster using LEACH-style algorithm.
 2. Compute routes from cluster-heads to base station (Dijkstra/A* or DP).
 3. Update energy consumption:
 - Member nodes: Transmit data to cluster-head.
 - Cluster-heads: Receive from members, transmit aggregated data to base station via route (including intermediate node energy for reception/transmission).
 4. Remove nodes with residual energy ≤ 0 .
 5. Log statistics: alive nodes, total residual energy, cluster-head list, routes.
- **Termination Conditions:** Network dies (no alive nodes) or maximum rounds reached.
- **Output:** Saves round-by-round log to CSV, prints real-time progress.

Visualization (`visualize.py`)

- Generates plots from simulation results:
- Network lifetime: Alive nodes vs rounds, total energy vs rounds (marks FND and HND).
- Routing trees: Network topology for selected rounds showing nodes, cluster-heads (red), members (blue), base station (yellow star), and routes to base station (green dashed lines).
- Future extensions: Animated energy heatmap, algorithm comparison charts.

2.3 Dependencies & Tools

- **Python 3.11+**
- **Libraries:**
 - `networkx` (for graph utilities, though core algorithms are custom)
 - `numpy` (numerical operations)
 - `matplotlib` (plotting and visualization)
 - `pytest` (unit testing framework)
- **Installation:** `pip install -r requirements.txt`

2.4 Testing

- Unit tests for each module:
- ``test_network.py``: Node creation, distance, energy consumption, graph building, alive nodes.
- ``test_energy_model.py``: Initialization, transmission (free space/multipath), reception, energy cost computation.
- ``test_clustering.py``: Basic clustering, energy-weighted selection, dead node handling, simulation round.
- ``test_routing.py``: Dijkstra/A* correctness, path format, cost matching, cluster-head routing, algorithm comparison.
- ``test_dp_lifetime.py``: Simple lifetime maximization, edge cases (dead nodes, no path).
- All tests pass, validating correctness of implementations.

3. Results

3.1 Simulation Execution

The simulator was run multiple times with varying configurations to validate functionality:

Default Test Run (``python main.py``):

- 50 nodes, 100m × 100m area, base station at (50, 50)
- Initial energy: 2.0 J per node
- Desired cluster ratio: 5% (≈ 2 cluster-heads per round)
- Routing algorithm: Dijkstra
- Rounds simulated: 50 (no node deaths due to conservative parameters)
- Output:
 - ``results/simulation_log.csv`` (50 rows of round data)
 - ``results/network_lifetime.png``
 - ``results/routing_tree_round_X.png`` for rounds 1, 50

Extended Run (``python run_simulation.py``):

- 50 nodes, 200 rounds
- Demonstrated stable operation with gradual energy depletion
- Final statistics after 200 rounds:
 - Alive nodes: 50 (all nodes still alive)
 - Total residual energy: 95.52 J (initial: 100.0 J)
 - Average energy dissipation per round: ~ 0.022 J/node
 - Cluster-heads per round: Consistently 2 (as expected from 5% of 50 nodes)

DP-Enabled Test (``test_dp_simulation.py``):

- Verified dynamic programming routing option functions correctly
- Smaller scale (20 nodes, 50 rounds) for quick validation

3.2 Generated Output Files

All requested outputs were successfully produced:

- ■ `simulation\_log.csv`: Round-by-round statistics (round number, alive nodes, total energy in joules)
- ■ `network\_lifetime.png`: Two subplots:
 - Top: Alive nodes vs rounds (flat line at 50 for test run)
 - Bottom: Total residual energy vs rounds (gradual linear decrease)
- ■ `routing\_tree\_round\_X.png`: For rounds 1, 50, 100, 200 (in 200-run):
 - Visualizes node positions (alive: blue, dead: gray, cluster-heads: red)
 - Shows base station location (yellow star)
 - Displays routes from each cluster-head to base station (green dashed lines)
 - Includes legend, grid, axis labels, and title with round number

3.3 Algorithm Validation

- **Correctness:** All unit tests pass, confirming:
 - Accurate distance and energy calculations
 - Proper clustering with energy-weighted head selection
 - Valid pathfinding (Dijkstra/A* produce identical optimal paths)
 - DP lifetime maximization selects optimal bottleneck paths
- **Efficiency:**
 - Dijkstra and A produce identical path costs (confirming A's admissible heuristic)
 - A* expands fewer nodes than Dijkstra in non-trivial topologies (as seen in test output: 5 nodes expanded vs Dijkstra's implicit full exploration)
- **Energy Model:** Parameters align with LEACH literature; transmission energy scales correctly with distance² (free space) and distance■ (multipath).

3.4 Code Quality & Documentation

- **Modularity:** Each major functionality in separate, well-encapsulated files.
- **Readability:** Clear docstrings, inline comments explaining complex logic (especially DP recurrence and routing energy consumption).
- **Standards Compliance:**
 - Type hints for function signatures
 - Meaningful variable and function names
 - Consistent indentation and formatting
 - Comprehensive README with usage instructions
- **Reproducibility:** Random seed not fixed (for realistic variation), but all parameters configurable via command line or direct modification.

4. Conclusions

The project successfully fulfills all requirements specified in the original prompt:

1. **Complete Implementation:** All seven modules developed from scratch with clear responsibilities.
2. **DSA Focus:** Core algorithms (graph traversal, heap-based priority queues, clustering, dynamic programming) are implemented without relying on library shortcuts for core logic.
3. **Energy-Aware Routing:** Simulation incorporates realistic energy consumption model, dynamic edge weights, and lifetime-aware routing options.
4. **Comparative Analysis:** Built-in comparison of Dijkstra and A* algorithms.
5. **Dynamic Programming:** Novel contribution for maximizing network lifetime via bottleneck optimization.
6. **Verification:** Rigorous unit testing ensures correctness of each component.
7. **Output & Visualization:** Simulation logs and plots generated as requested.
8. **Documentation:** Clear instructions and explanations provided for reuse and evaluation.

The simulation demonstrates stable WSN operation under the configured parameters, with energy depletion occurring gradually over time. The modular design allows easy extension for future work (e.g., more realistic data aggregation, alternative routing protocols, or enhanced visualization).

5. References

1. W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, 2000.
2. W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "An application-specific protocol architecture for wireless microsensor networks," *IEEE Transactions on Wireless Communications*, vol. 1, no. 4, pp. 660-670, Oct. 2002.
3. C. Perkins, E. M. Royer, and S. Das, *Ad Hoc On-Demand Distance Vector (AODV) Routing*, RFC 3561, July 2003. (For routing context)
4. S. Bandyopadhyay and E. J. Coyle, "An energy efficient hierarchical clustering algorithm for wireless sensor networks," in *Proceedings of the 22nd Annual Joint Conference of the IEEE Computer and Communications Societies*, 2003.

Appendix: Files Generated

- **Source Code:** `src/` directory (all .py files)
- **Tests:** `test\_\*.py` scripts (verification suite)
- **Documentation:**

- `README.md` (user instructions)
- `FINAL\_SUMMARY.md` (detailed technical summary)
- `PROJECT\_COMPLETE.md` (completion notice)
- `first\_review\_report.md` (this document)
- **Results:** `results/` directory (simulation logs and plots)
- **Requirements:** `requirements.txt` (dependency list)

End of Report